claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-abff1f46ab1f06ee0.jsonl`
- SHA-256: `5a60a92dea4f17bcdbbb7db1b5da5dd2106f0d1f545a1e42b4530893545d85a8`
- Source modified: `2026-07-02T03:01:38+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T03:00:17.744Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is
2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-
guru/badminton-highlight-indexer:

{
  "title": "Stage-2 AI-handoff orchestration duplicated between the default segmenter
(yolo_hybrid) and the trajectory family",
  "category": "tech-debt",
  "severity": "P2",
  "file": "backend/pipeline/segmenters/trajectory_hybrid.py",
  "line": 390,
  "evidence": "trajectory_hybrid.py:390-440 process_candidate_window is a near-copy of
yolo_hybrid.py:334ff ? same window padding, extract_video_chunk(reencode=True),
sport_profile.get_prompt, provider.analyze_video, timestamp re-offsetting, cleanup,
ThreadPoolExecutor fan-out, and the identical 'B2: a provider failure returns [] with
metrics[error] set' comment block in both files. _fmt_ts is duplicated verbatim
(yolo_hybrid.py:35-41 vs trajectory_hybrid.py:40-45). Both paths are LIVE: yolo_hybrid is the
default segmenter (backend/config/models.py:150 default_segmenter=\"yolo_hybrid\") and the
trajectory family (TrajectoryHybridSegmenter + Fusion/TrackNet/Wasb subclasses,
fusion_hybrid.py:41 etc.) is the fusion path. A bug fix to handoff semantics (e.g. the B2 error
contract) must be applied twice and has already started drifting (yolo_hybrid keeps a separate
_run_ai_handoff/_persist_candidates split, yolo_hybrid.py:262-404).",
  "proposed_fix": "Extract a shared handoff helper (e.g. backend/pipeline/segmenters/handoff.py:
pad_window + process_candidate_window + the executor fan-out) consumed by both
HybridYoloSegmenter and TrajectoryHybridSegmenter; move _fmt_ts to backend/utils. Note the
trajectory subclass family itself (fusion/tracknet/wasb via _build_runner template methods) is
well-factored ? the duplication is only across the yolo/trajectory family boundary.",
  "effort": "M",
  "tracked_in": ""
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?
```

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:00:25.774Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20
21      import concurrent.futures
22      import logging
23      import os
24      import time
25      from typing import Any, Dict, List, Optional, Tuple
26
27      import cv2
28
29      from backend.config.models import IndexingConfig
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.segmenters.base import BasePlaySegmenter
32      from backend.providers import provider_registry
33      from backend.sports import sport_registry
34      from backend.utils.paths import output_base
35      from backend.utils.video import extract_video_chunk, get_video_duration
36
37      logger = logging.getLogger(__name__)
38
39
40      def _fmt_ts(seconds: float) -> str:
41          seconds = max(0, int(seconds))
42          h, rem = divmod(seconds, 3600)
43          m, s = divmod(rem, 60)
44          return f"{h:02d}:{m:02d}:{s:02d}"
45
46
47      class TrajectoryHybridSegmenter(BasePlaySegmenter):
48          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
49
50          #: config section under `indexing` (velocity_threshold lives there), the
51          #: output subdir for trajectories, and the metadata detector-tag prefix.
52          family: str = ""
53          #: human-readable label used in log lines.
54          display_label: str = ""
55
56          def _build_runner(
57              self, idx_cfg: IndexingConfig
```

```
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
            raise NotImplementedError

        def _build_native_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
            native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
            raise NotImplementedError(
                f"detector_impl='native' is not supported for the "
                f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
                f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
            )

        def _resolve_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.

            ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
            ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
            is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
            in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
            ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
            (the WSL runner) so the default production path is unchanged
            (``docs/DECOUPLED_COMPUTE/``).
            """
            # Safety net: accept raw dicts from legacy / test callers.
            if isinstance(idx_cfg, dict):
                idx_cfg = IndexingConfig(**idx_cfg)
            impl = (
                os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
            ).lower()
            if impl == "stub":
                from backend.pipeline.detectors.stub_runner import (
                    StubConfig,
                    StubDetectorRunner,
                )

                logger.info(
                    "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                    self.display_label or "trajectory",
                )
                return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
                    "detector_impl": "stub"
                }
            if impl in ("native", "native_wasb", "wasb_native"):
                logger.info(
                    "%s: detector_impl=native ? Linux-native runner (no WSL).",
                    self.display_label or "trajectory",
                )
                return self._build_native_runner(idx_cfg)
            return self._build_runner(idx_cfg)

        @staticmethod
```

```
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(
            self,
            cw: Dict[str, Any],
            points: List[Any],
            fps: float,
            frame_width: float,
            video_path: str,
            idx_cfg: "IndexingConfig",
        ) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(
            self,
            windows: List[Dict[str, Any]],
            window_stats: List[Dict[str, Any]],
            idx_cfg: "IndexingConfig",
        ) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(  # type: ignore[override]
            self,
            video_id: str,
```

```
            video_path: str,
            player_pool: Optional[List[str]] = None,
            max_frames: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.sport
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.indexing
            skip_ai = bool(idx_cfg.skip_ai_handoff)
            provider_name = self.config.ai_provider
            if skip_ai:
                model_name = "local-noai"
                logger.info(
                    "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                    "be emitted as rallies; no %s handoff, no API key needed.",
                    label,
                    provider_name,
                )
            else:
                model_name = self.config.ai_model
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint

                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(
                        provider_name, api_key=api_key, model_name=model_name
                    )
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            try:
                runner, runner_params = self._resolve_runner(idx_cfg)
            except NotImplementedError as e:
                # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
                # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
```

```
237                 logger.error("%s: %s", label, e)
238                 return []
239
240             velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
241             merge_gap = idx_cfg.dead_time_merge_gap
242             min_window_duration = idx_cfg.min_action_window_duration
243             # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
244             max_window_duration = idx_cfg.max_window_duration
245             window_min_split_gap = idx_cfg.window_min_split_gap
246             window_hard_cap = idx_cfg.window_hard_cap
247             window_inactivity_end = idx_cfg.window_inactivity_end
248             inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
249             inactivity_min_density = idx_cfg.inactivity_min_density
250             inactivity_temporal_density = idx_cfg.inactivity_temporal_density
251             window_blob_fallback = idx_cfg.window_blob_fallback
252             window_blob_factor = idx_cfg.window_blob_factor
253             handoff_padding = idx_cfg.ai_handoff_padding
254             ai_max_workers = idx_cfg.ai_max_workers
255             log_candidates = idx_cfg.log_yolo_candidates
256             store_candidates = idx_cfg.store_yolo_candidates
257
258             detection_params = {
259                 "detector": self.name,
260                 "velocity_thresh": velocity_thresh,
261                 "merge_gap": merge_gap,
262                 "min_action_window_duration": min_window_duration,
263                 **runner_params,
264             }
265
266             # --- Phase 1: env healthcheck (fail fast with a clear message) ---
267             ok, msg = runner.healthcheck()
268             if not ok:
269                 logger.error("%s detector environment not ready: %s", label, msg)
270                 return []
271             logger.info("%s detector env OK: %s", label, msg)
272
273             # --- Phase 2: trajectory -> candidate rally windows ---
274             # Trajectory detectors process the whole video in one pass (they carry
275             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
276             t_start = time.time()
277             out_dir = os.path.join(output_base(self.config), self.family)
278             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
279             if not csv_path:
280                 logger.error("%s produced no trajectory; aborting.", label)
281                 return []
282
283             points = runner.parse_trajectory_csv(csv_path)
284             logger.info(
285                 "Parsed %d trajectory points (%d visible).",
286                 len(points),
287                 sum(1 for p in points if p.visible),
288             )
289
290             all_candidate_windows = runner.trajectory_to_action_windows(
291                 points,
292                 fps=fps,
293                 frame_width=frame_width,
294                 chunk_start=0.0,
295                 velocity_thresh=velocity_thresh,
296                 merge_gap=merge_gap,
297                 min_window_duration=min_window_duration,
298                 chunk_index=0,
299                 max_window_duration=max_window_duration,
300                 min_split_gap=window_min_split_gap,
```

```
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                inactivity_temporal_density=inactivity_temporal_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info(
                "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                label,
                time.time() - t_start,
                len(all_candidate_windows),
            )

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(
                        f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                        f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
                    )

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(
                    cw, points, fps, frame_width, video_path, idx_cfg
                )
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id,
                        detector=self.name,
                        start_time=cw["start"],
                        end_time=cw["end"],
                        chunk_index=cw["chunk_index"],
                        confidence=min(1.0, cw["peak_velocity"]),
                        stats=window_stats[i],
                        detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(
                all_candidate_windows, window_stats, idx_cfg
            )

            ai_segments: List[dict] = []
            if skip_ai:
```

```
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info(
                    "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
                    "(no AI handoff).",
                    len(ai_segments),
                )
            else:
                handoff_dir = os.path.join(
                    output_base(self.config), f"chunks_{provider_name}_handoff"
                )
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info(
                    "Phase 3: %s validation on %d candidate windows...",
                    provider_name,
                    len(handoff_indices),
                )

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(
                        video_path, w_start, w_dur, w_path, reencode=True
                    ):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, metrics = provider.analyze_video(
                        w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
                    )
                    # B2: a provider failure returns [] with metrics["error"] set ? that is
                    # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
                    # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
                    if metrics.get("error"):
                        logger.warning(
                            "[Handoff %d] provider error ? window %d skipped: %s",
                            wi + 1,
                            wi,
                            metrics["error"],
                        )
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
```

```
                            logger.warning(
                                "Dropping segment with unparseable timestamps: %s ? %s",
                                seg,
                                e,
                            )
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(
                    max_workers=ai_max_workers
                ) as executor:
                    futures = [
                        executor.submit(
                            process_candidate_window, wi, all_candidate_windows[wi]
                        )
                        for wi in handoff_indices
                    ]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info(
                    "Phase 3 completed. AI confirmed %d semantic play segments.",
                    len(ai_segments),
                )

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
                    )
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
                                "duration": dur,
                                "metadata": metadata,
                            }
                        )
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)

            return segments_data
```

3. user (2026-07-02T03:00:26.872Z)

```
"""Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
(Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).

Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
(`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
can be consumed by the future fusion segmenter; this segmenter just orchestrates
chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.

Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
detection uses torchvision's permissively-licensed (BSD) detectors and the
association/tracking that YOLO.track() used to bundle in is provided by
supervision's MIT-licensed ByteTrack.

The registry key stays "yolo_hybrid" for back-compat with existing configs and the
DB `detector` tags, even though no YOLO is involved anymore.
"""

import concurrent.futures
import logging
import os
import time
from typing import Any, Dict, List, Tuple

from backend.pipeline.detectors.person_detector import PersonDetector
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.providers import provider_registry
from backend.sports import sport_registry
from backend.utils.paths import output_base
from backend.utils.video import extract_video_chunk, get_video_duration

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class HybridYoloSegmenter(BasePlaySegmenter):
    def __init__(self, db, config):
        super().__init__(db, config)
        # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
        # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
        self.person = PersonDetector(config)

    @property
    def name(self) -> str:
        return "yolo_hybrid"

    @property
    def description(self) -> str:
        return (
            "Two-stage pipeline: torchvision person detection + ByteTrack for fast "
            "candidate filtering, then an AI provider for high-precision semantic
boundaries."
        )
```

```
60
61          def _resolve_provider(self, video_path: str):
62              """Phase 0: resolve sport profile + AI provider (with API key).
63
64              Returns the configured provider instance, or ``None`` to signal that
65              ``process_video`` should early-return ``[]`` (bad sport, missing API key,
66              or unknown provider). Behaviour is identical to the inlined version.
67              """
68              # Get sport profile config
69              sport_name = self.config.sport
70              try:
71                  sport_registry.get(sport_name)
72              except KeyError as e:
73                  logger.error(str(e))
74                  return None
75
76              # Get AI provider and model config
77              provider_name = self.config.ai_provider
78              model_name = self.config.ai_model
79
80              # Get appropriate API key based on the provider
81              api_key_env_var = f"{provider_name.upper()}_API_KEY"
82              api_key = os.environ.get(api_key_env_var)
83              if not api_key:
84                  from backend.pipeline.segmenters._keyhint import api_key_hint
85
86                  logger.error(
87                      f"{api_key_env_var} environment variable is not set! "
88                      f"To run the {provider_name} segmenter, set your API key. "
89                      + api_key_hint(api_key_env_var)
90                  )
91                  return None
92
93              try:
94                  provider = provider_registry.get(
95                      provider_name, api_key=api_key, model_name=model_name
96                  )
97              except KeyError as e:
98                  logger.error(str(e))
99                  return None
100
101             logger.info(
102                 f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
103             )
104             return provider
105
106         def _collect_candidate_windows(
107             self,
108             video_id: str,
109             video_path: str,
110             video_duration: float,
111             idx_cfg: Dict[str, Any],
112             chunk_duration: float,
113             chunk_overlap: float,
114             velocity_thresh: float,
115             merge_gap: float,
116             frame_skip: int,
117             chunks_dir: str,
118         ) -> List[Dict[str, Any]]:
119             """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
120             sequential), then dedup-merge overlapping windows across chunks. Verbatim
move."""
121             step = chunk_duration - chunk_overlap
122             chunk_starts = []
```

```
            t = 0.0
            while t < video_duration:
                chunk_starts.append(t)
                t += step

            num_chunks = len(chunk_starts)
            logger.info(
                f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks."
            )

            all_candidate_windows = []

            t_start = time.time()
            prefetch_workers = idx_cfg.yolo_prefetch_workers

            if num_chunks > 1 and prefetch_workers > 0:
                # --- PIPELINED PROCESSING ---
                # GPU inference must stay sequential (single GPU), but ffmpeg chunk
                # extraction is pure I/O. We pre-extract upcoming chunks in background
                # threads so extraction of chunk N+1 overlaps with inference on chunk N.
                logger.info(
                    f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)..."
                )

                extract_executor = concurrent.futures.ThreadPoolExecutor(
                    max_workers=prefetch_workers
                )

                def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
                    chunk_end = min(cs + chunk_duration, video_duration)
                    actual_dur = chunk_end - cs
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
                    success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
                    return (ci, cs, chunk_path, success)

                # Submit all extractions upfront ? they'll queue and run as workers free up
                extract_futures = [
                    extract_executor.submit(_extract_chunk, ci, cs)
                    for ci, cs in enumerate(chunk_starts)
                ]

                # Process results in order: wait for extraction, then run inference on GPU
                for future in extract_futures:
                    ci, chunk_start, chunk_path, success = future.result()
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    logger.info(
                        f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
                    )

                    if not success:
                        logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path,
                        chunk_start,
                        velocity_thresh,
                        merge_gap,
                        frame_skip=frame_skip,
                        chunk_index=ci,
                    )
                    logger.info(
```

```
                        f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
                    )
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                extract_executor.shutdown(wait=False)
            else:
                # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
                for ci, chunk_start in enumerate(chunk_starts):
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_dur = chunk_end - chunk_start
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    logger.info(
                        f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
                    )
                    success = extract_video_chunk(
                        video_path, chunk_start, actual_dur, chunk_path
                    )
                    if not success:
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path,
                        chunk_start,
                        velocity_thresh,
                        merge_gap,
                        frame_skip=frame_skip,
                        chunk_index=ci,
                    )
                    logger.info(
                        f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
                    )
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb)
                    / total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

            if all_candidate_windows:
                all_candidate_windows.sort(key=lambda w: w["start"])
                merged_windows = [all_candidate_windows[0]]
```

```
                for current in all_candidate_windows[1:]:
                    last = merged_windows[-1]
                    if current["start"] <= last["end"]:  # Overlap
                        merged_windows[-1] = _merge_stats(last, current)
                    else:
                        merged_windows.append(current)
                all_candidate_windows = merged_windows

            logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
            logger.info(f"Total candidate windows: {len(all_candidate_windows)}")

            return all_candidate_windows

        def _persist_candidates(
            self,
            video_id: str,
            all_candidate_windows: List[Dict[str, Any]],
            detection_params: Dict[str, Any],
            log_candidates: bool,
            store_candidates: bool,
        ) -> List[Any]:
            """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
            dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
move."""
            # Per-rally console log (final, full-video timestamps) for video cross-
verification.
            if log_candidates:
                for w in all_candidate_windows:
                    logger.info(
                        f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
                        f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
                        f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
                        f"tracks={w['track_id_count']}"
                    )

            # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
            # so confirmed AI segments can be linked back in Phase 4.
            candidate_ids = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, w in enumerate(all_candidate_windows):
                    stats = {
                        "peak_velocity": w["peak_velocity"],
                        "mean_velocity": w["mean_velocity"],
                        "active_frame_count": w["active_frame_count"],
                        "track_id_count": w["track_id_count"],
                    }
                    # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
                    confidence = min(1.0, w["peak_velocity"])
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id,
                        detector="yolo_hybrid",
                        start_time=w["start"],
                        end_time=w["end"],
                        chunk_index=w["chunk_index"],
                        confidence=confidence,
                        stats=stats,
                        detection_params=detection_params,
                    )

            return candidate_ids

        def _run_ai_handoff(
```

```
            self,
            video_id: str,
            video_path: str,
            video_duration: float,
            all_candidate_windows: List[Dict[str, Any]],
            candidate_ids: List[Any],
            provider,
            sport_profile,
            provider_name: str,
            chunk_duration: float,
            handoff_padding: float,
            ai_max_workers: int,
        ) -> List[dict]:
            """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
            the provider for semantic boundaries, in parallel. Verbatim move."""
            gemini_segments = []
            gemini_dir = os.path.join(
                output_base(self.config), f"chunks_{provider_name}_handoff"
            )
            os.makedirs(gemini_dir, exist_ok=True)

            logger.info(
                f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
            )

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start

                w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
                # Re-encode short handoff clips to avoid keyframe-boundary drift
                success = extract_video_chunk(
                    video_path, w_start, w_dur, w_path, reencode=True
                )
                if not success:
                    return []

                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, metrics = provider.analyze_video(
                    w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
                )
                # B2: a provider failure returns [] with metrics["error"] set ? that is
                # "we don't know", NOT "no rallies in this window". Surface it loudly so a
                # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
                if metrics.get("error"):
                    logger.warning(
                        "[Handoff %d] provider error ? window %d skipped: %s",
                        wi + 1,
                        wi,
                        metrics["error"],
                    )

                # Coordinate mapping ? shift local timestamps back to full-video time
                valid_segments = []
                for segment in segments:
                    try:
                        segment["start_time"] = float(segment["start_time"]) + w_start
                        segment["end_time"] = float(segment["end_time"]) + w_start
                        # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
                        segment["_candidate_id"] = candidate_ids[wi]
                        valid_segments.append(segment)
```

```
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning(
                            f"Dropping segment with unparseable timestamps: {segment} ? {e}"
                        )

                try:
                    os.remove(w_path)
                except OSError:
                    pass

                return valid_segments

            with concurrent.futures.ThreadPoolExecutor(
                max_workers=ai_max_workers
            ) as executor:
                futures = [
                    executor.submit(process_candidate_window, wi, w)
                    for wi, w in enumerate(all_candidate_windows)
                ]
                # Iterate in submission order to preserve temporal ordering
                for future in futures:
                    gemini_segments.extend(future.result())

            try:
                os.rmdir(gemini_dir)
            except OSError:
                pass

            logger.info(
                f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
            )
            return gemini_segments

        def _populate_db(
            self, video_id: str, gemini_segments: List[dict], model_name: str
        ) -> List[Dict[str, Any]]:
            """Phase 4: DB Population ? write confirmed AI segments, link them back to their
            source candidate, and build the returned segments_data. Verbatim move."""
            segments_data = []
            for segment in gemini_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start

                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None:
                        shot_count = max(3, int(dur / 0.8))
                    else:
                        try:
                            shot_count = int(shot_count)
                        except (ValueError, TypeError):
                            shot_count = max(3, int(dur / 0.8))

                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"yolo-hybrid-{model_name}",
                    }

                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
```

```
                    )
                    if segment_id:
                        # Link back to the candidate that produced this segment. A candidate
                        # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
                        # the last one ? the link table records "this candidate was
confirmed".
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
                                "duration": dur,
                                "metadata": metadata,
                            }
                        )
                except Exception as e:
                    logger.warning(f"Skipping invalid segment struct: {e}")

            return segments_data

        def process_video(
            self,
            video_id: str,
            video_path: str,
            player_pool: List[str] = None,
            max_frames: int = None,
        ) -> List[Dict[str, Any]]:
            self.person.lazy_load_model()

            # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
            provider = self._resolve_provider(video_path)
            if provider is None:
                return []

            # Re-read the config values needed downstream (pure deterministic reads).
            idx_cfg = self.config.indexing
            sport_profile = sport_registry.get(self.config.sport)
            provider_name = self.config.ai_provider
            model_name = self.config.ai_model

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []

            chunk_duration = idx_cfg.chunk_duration
            chunk_overlap = idx_cfg.chunk_overlap
            velocity_thresh = (
                idx_cfg.yolo_velocity_threshold
            )  # Fraction of frame width per second
            merge_gap = idx_cfg.dead_time_merge_gap
            frame_skip = idx_cfg.yolo_frame_skip
            tracker_config = idx_cfg.yolo_tracker
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            # Snapshot of the params that produced these detections ? stored alongside each
            # candidate so a fine-tuning run can reproduce / correlate against them.
```

```
            detection_params = {
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "frame_skip": frame_skip,
                "tracker": tracker_config,
                "detector_model": idx_cfg.detector_model,
                "detector_score_threshold": idx_cfg.detector_score_threshold,
                "min_action_window_duration": idx_cfg.min_action_window_duration,
            }

            chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
            os.makedirs(chunks_dir, exist_ok=True)

            # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
            all_candidate_windows = self._collect_candidate_windows(
                video_id,
                video_path,
                video_duration,
                idx_cfg,
                chunk_duration,
                chunk_overlap,
                velocity_thresh,
                merge_gap,
                frame_skip,
                chunks_dir,
            )

            # Phase 2b: log + persist raw candidates (window index -> candidate_id).
            candidate_ids = self._persist_candidates(
                video_id,
                all_candidate_windows,
                detection_params,
                log_candidates,
                store_candidates,
            )

            try:
                os.rmdir(chunks_dir)
            except Exception:
                pass

            # Phase 3: AI Provider Handoff
            gemini_segments = self._run_ai_handoff(
                video_id,
                video_path,
                video_duration,
                all_candidate_windows,
                candidate_ids,
                provider,
                sport_profile,
                provider_name,
                chunk_duration,
                handoff_padding,
                ai_max_workers,
            )

            # Phase 4: DB Population
            return self._populate_db(video_id, gemini_segments, model_name)


    # Register the segmenter
    segmenter_registry.register(HybridYoloSegmenter)

```

4. assistant (2026-07-02T03:00:34.225Z)

Both cited files confirm the duplication. Now let me verify the config default, the fusion subclass, check git history for recent fixes, and see whether a shared handoff helper already exists.

5. user (2026-07-02T03:00:37.207Z)

```
cc60ebe Fix/rally detection edge cases (#394)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
5b71af8 D3-c: remove AppConfig .get() shim, flip extra='forbid' (#367)
8b3694e refactor: segmenters use typed AppConfig access (D3-b) (#366)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
2afd54f feat(serving): M3 ? configurable output/scratch base (de-hardcode output/, default-OFF)
(#282)
dc53db5 feat(detectors): Linux-native WASB runner (M3.5) + worker real-trajectory extraction
(#256)
f06c6a6 fix: cross-platform canonicalization (Linux/macOS, not just Windows) (#252)
2f19e5f feat(detectors): CPU stub trajectory runner ? "mock a GPU with a CPU" (#247)
8092a39 refactor(segmenter): decompose yolo_hybrid process_video into phase helpers (#227)
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

6. user (2026-07-02T03:00:41.895Z)

```
147-
148-
149-class IndexingConfig(BaseModel):
150:    default_segmenter: str = "yolo_hybrid"
151-    use_gpu: bool = True
152-    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
153-    # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
--
241-    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
242-    fusion: FusionConfig = Field(default_factory=FusionConfig)
243-
244:    @field_validator("default_segmenter")
245-    @classmethod
246-    def segmenter_must_be_registered(cls, v: str) -> str:
247-        # Lazy import: the segmenter registry imports modules that may import
```

7. user (2026-07-02T03:00:42.140Z)

```
No files found
```

8. user (2026-07-02T03:00:48.096Z)

```
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py
badminton-highlight-indexer\backend\pipeline\segmenters\base.py
badminton-highlight-indexer\backend\pipeline\segmenters\_keyhint.py
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py
badminton-highlight-indexer\backend\pipeline\segmenters\motion.py
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py
badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py
```

9. user (2026-07-02T03:00:49.835Z)

```
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py:41:class
FusionSegmenter(TrajectoryHybridSegmenter):
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py:23:class
TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py:25:class
WasbHybridSegmenter(TrajectoryHybridSegmenter):
```

10. user (2026-07-02T03:00:59.459Z)

```
badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py:40:def
_fmt_ts(seconds: float) -> str:
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py:35:def _fmt_ts(seconds:
float) -> str:
badminton-highlight-indexer\backend\eval\harness.py:94:def _fmt_ts(seconds: float) -> str:
```

11. user (2026-07-02T03:01:38.569Z)

```
Structured output provided successfully
```